



Microcontroller Project

By

Zhixin, Suber and Zohaib

Aim

This project was created to develop testing equipment that it can be use in thermal tests, this is so that it can be possible to record and spectate heat dissipation at two different areas.

Workload

Zhixin

- Simulink
- MATLAB
- AFE Circuit and Simulation
- Presentation Slides
- Report Writing

Suber

- MPLABX (Assembly Code)
- PICSIM
- Presentation Slides
- Report Writing

Zohaib

- Fusion 360
- PCB Schematics
- Report Writing
- Presentation Slides

Requirements

Hardware

- Power Supply (9V)
- Power Regulation Circuit
- 2 NTC Thermistors
- AFE Circuit
- 40-Pin PIC (ADC) with electronic switch
- LCD Display

Software

- Matlab/Simulink
- MPLABX
- PICSIM
- Fusion 360

Overview of Tools

- MATLAB/Simulink - To create a layout of the AFE circuit
- Fusion 360 - To create PCB schematics
- MPLABX - Where the software was created, ran and tested
- PICSIM - Where the board could be simulated to test the code

Simulink

We used software related to MATLAB to create a virtual model of a physical system such as our AFE circuit.

The practical usage of this software is to design a schematic for our project before making the actual circuitry.

Fusion 360

Fusion 360 is the software we used to design a schematic of our PCB as we utilising the PCB layout tools to place components and route connections between them (Autodesk).

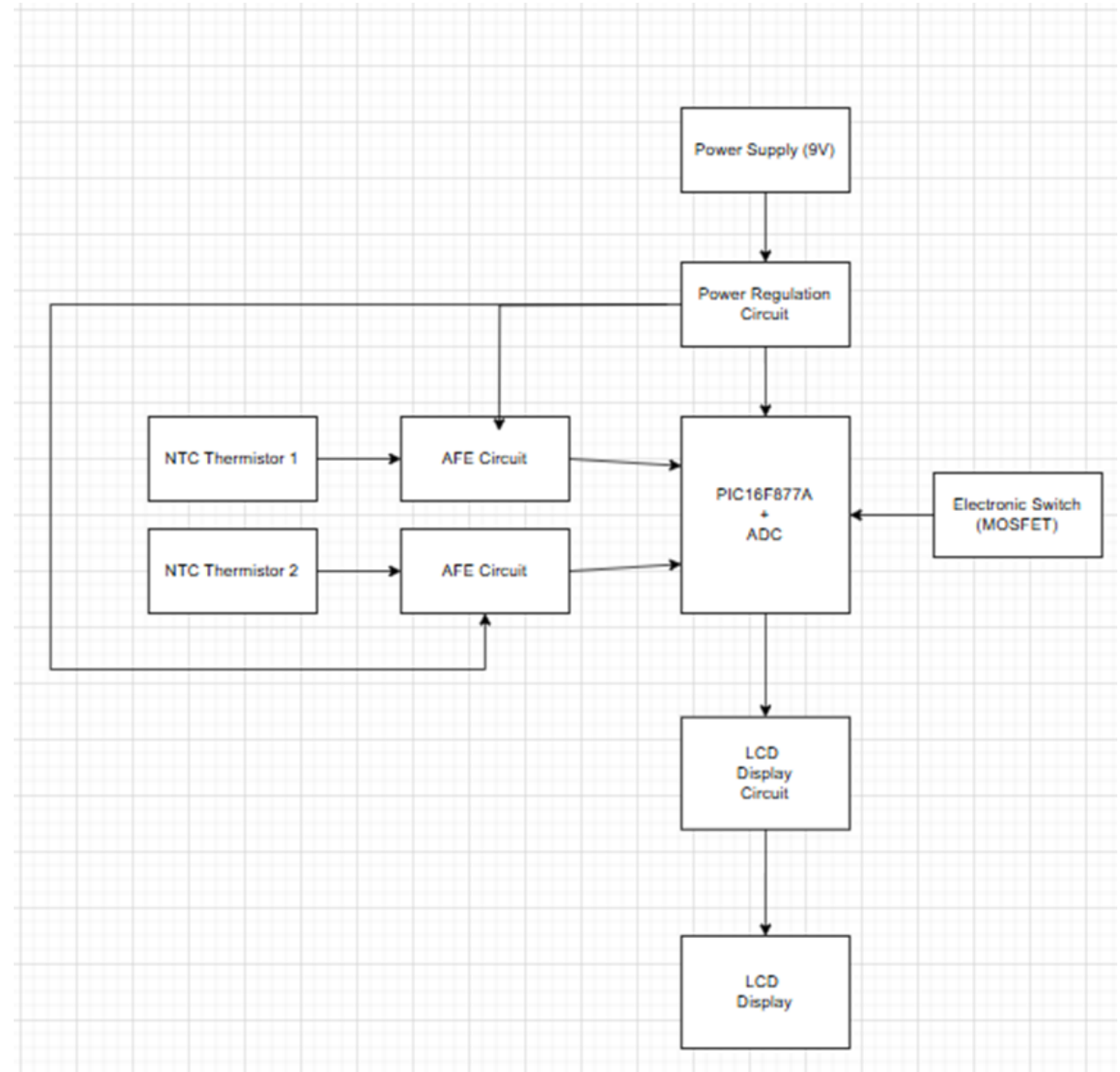
This allowed us to create a working prototype and make any changes easier, faster and at no cost.

MPLABX and PICSIM

MPLABX allowed us to develop and debug the code for our microcontroller, we used tools such as the logic analyser or the target memory view alongside breakpoints to analyse parts of our code.

After we exported the software as a hex file to PICSIM to simulate an actual board without hardware.

Project Overview



NTC Thermistors

Thermistors are resistors which resistance is dependent on the temperature of the environment around it, this is great for a data acquisition system (dwyeromega).

We chose to use NTC (Negative Temperature Coefficients) as our way to collect the analogue information we need. This means that as temperature increases the resistance decreases, so when temperature increases the voltage will go down (dwyeromega).

Analogue to Digital Converter (ADC)

The PIC that we are using has an ADC which is how we are convert our analogue information into digital signals so that it can be used in computer software (Arrow,2023).

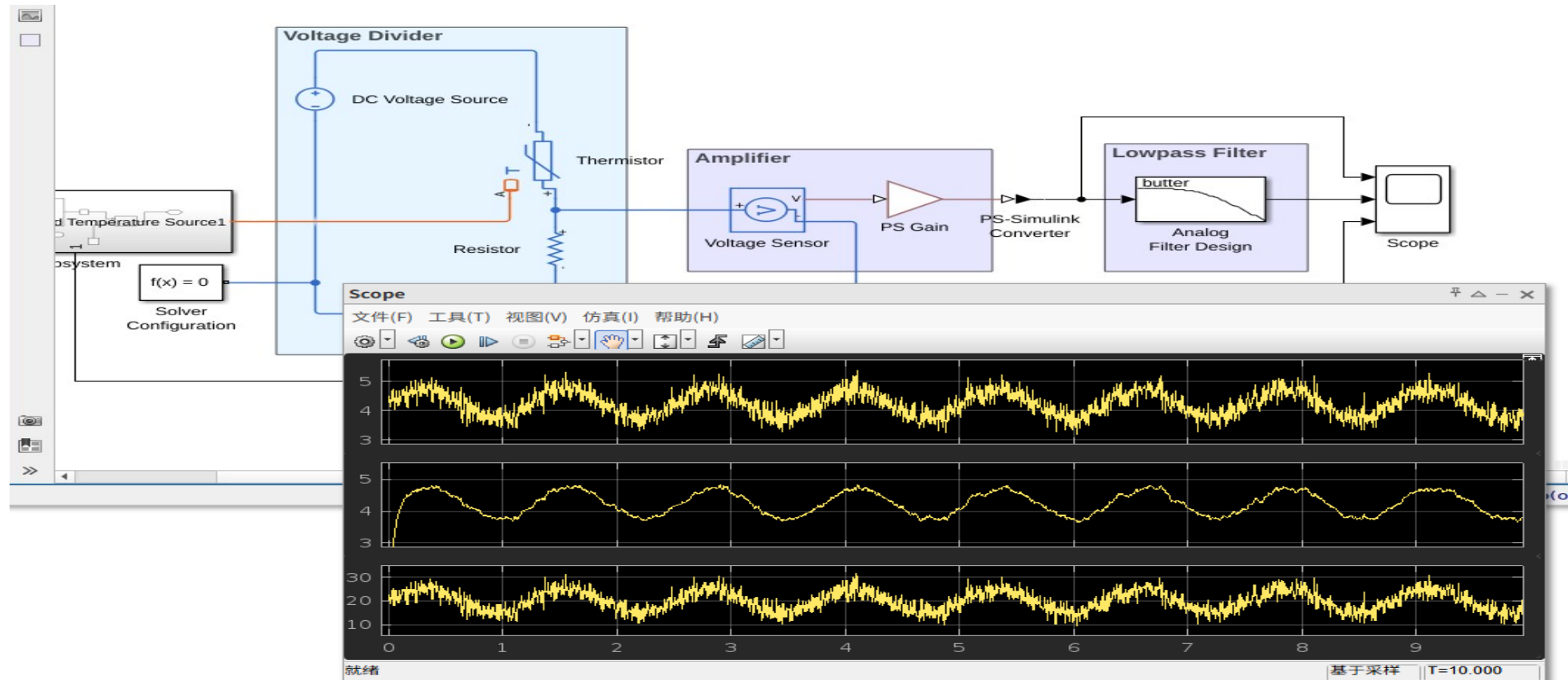
Digital signals are a sequence of discrete values whilst analogue signals are sequence of continuous values, ADC's sample and quantify the signal so it can be read by software.(Arrow, 2023).

Analogue Front End (AFE)

Analogue Front End (AFE) refers to a group of circuits located between the sensor and the subsequent digital processing unit for signal conditioning and pre-processing (Morris, 2001). Its main purpose is to convert the weak, noisy sensor signal to meet the system requirement, with sufficient signal-to-noise ratio and linearity of the analogue signal (Park and Mackay, 2003).

MATLAB and Simulink

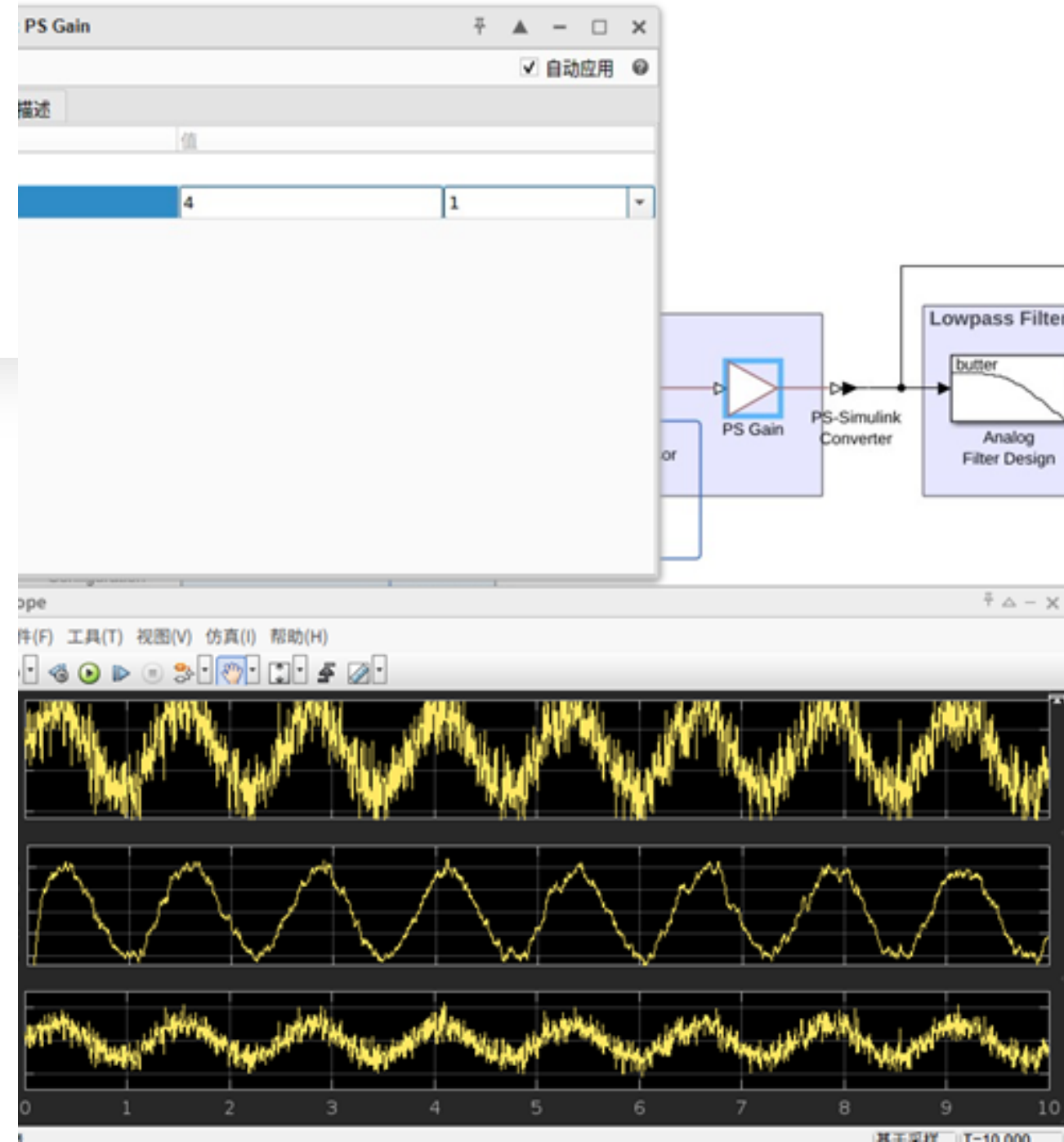
First, we ran the original circuit and observed the image



Change the Gain

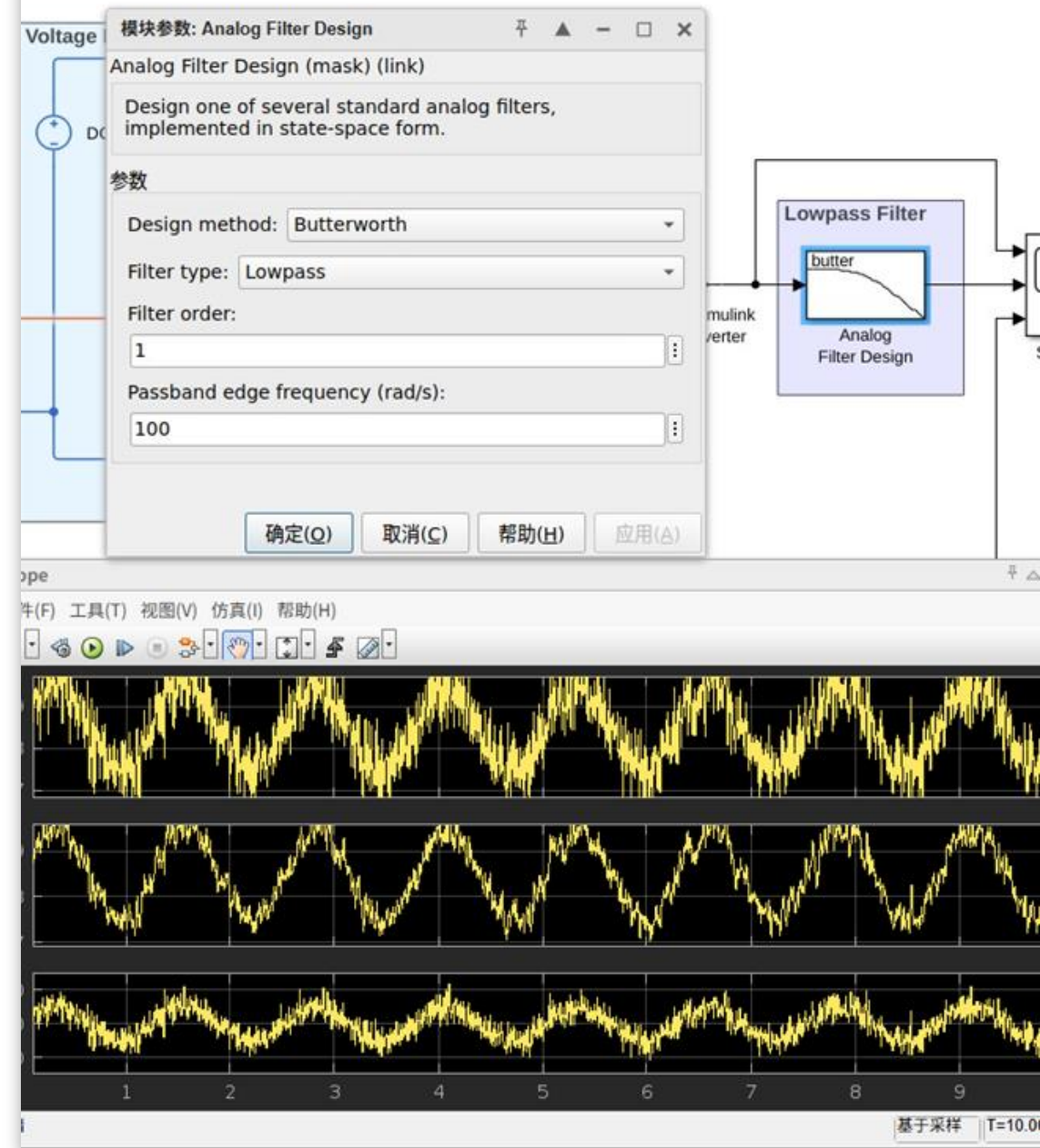
We changed the value of PS gain

We found that the greater the gain, the greater the amplitude

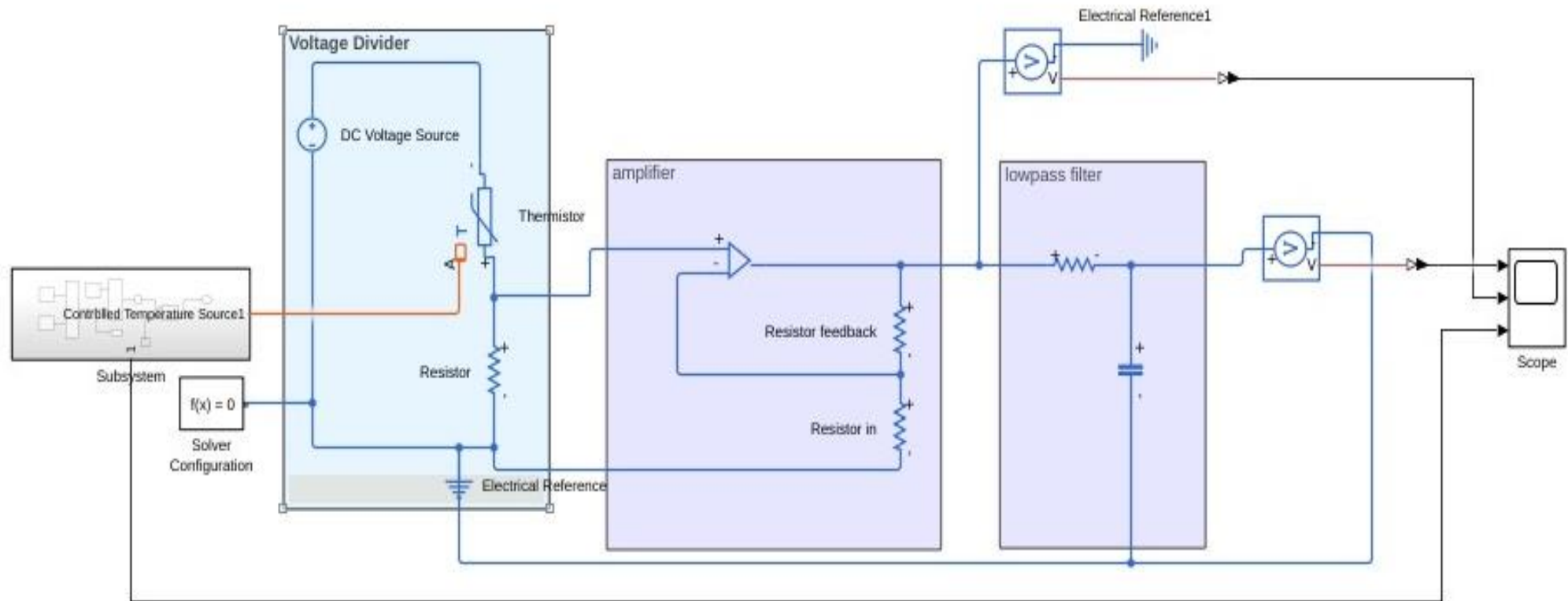


Changing Filter Parameters

The higher the passband edge frequency, the less effective the filter



Schematic of AFE Circuit



Why we used a non-inverting amplifier with an RC filter?

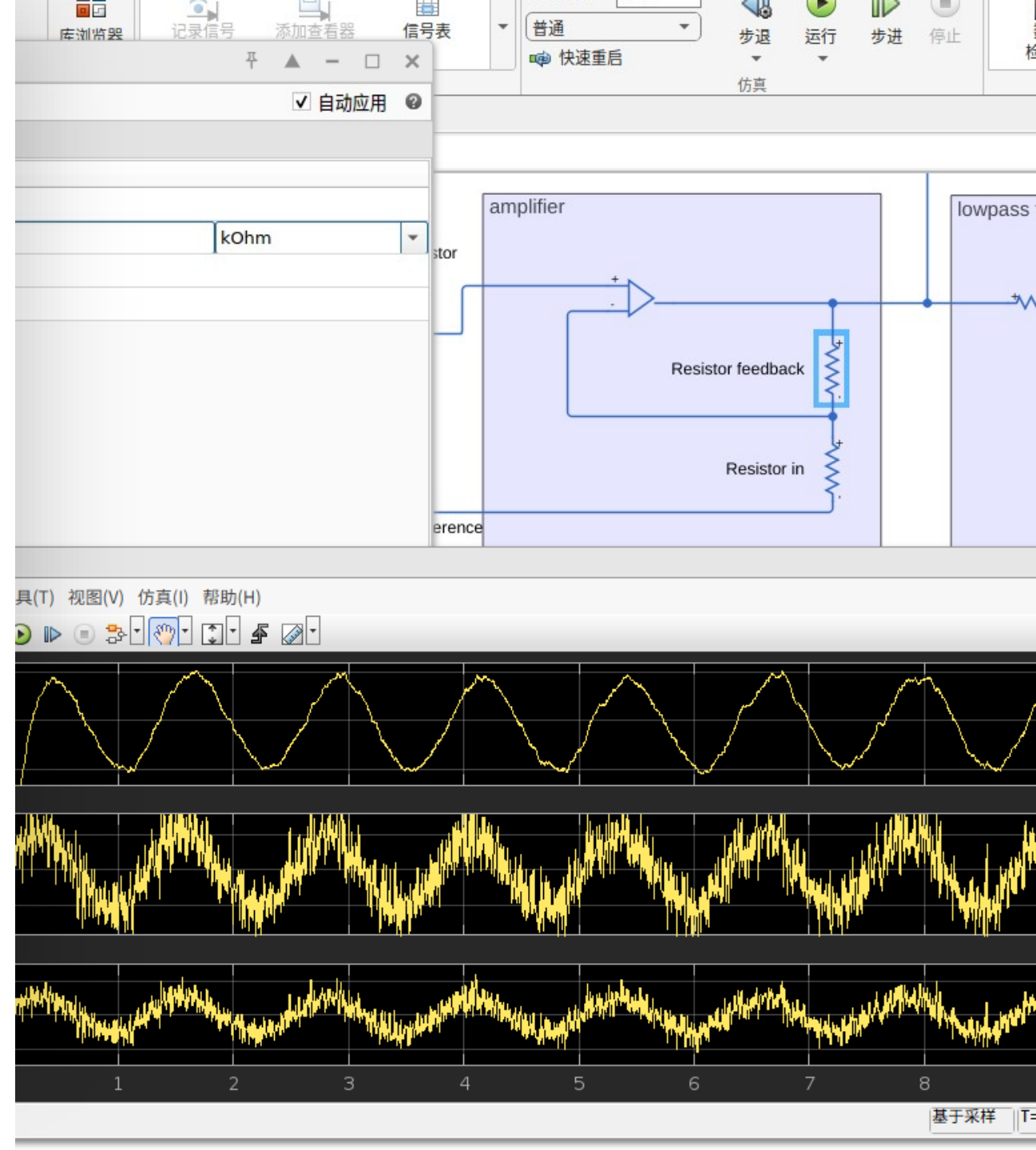
The circuit is simple and there is only one amplifier with a few resistors and capacitors, the number of components is small.

The gain is determined by R_f/R_i . The filters are determined by resistance and capacitance, the two do not interfere with each other and it can be optimised step by step during design.

Change the value

$$G=1+R_f/R_i$$

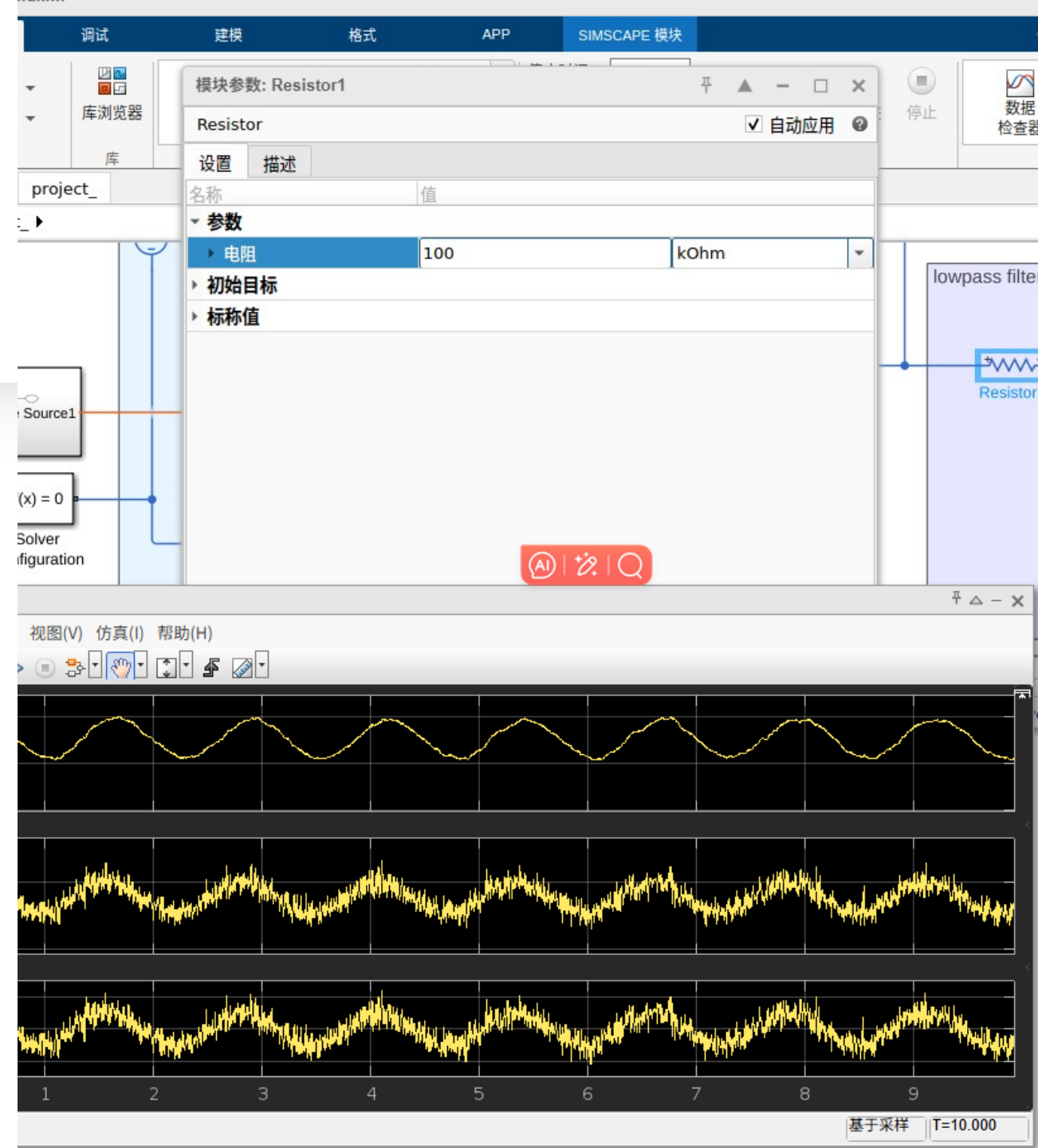
We can obtain the appropriate value of R_f based on this formula. we change the resistance value of R_f . The R_f is larger, the gain will be greater



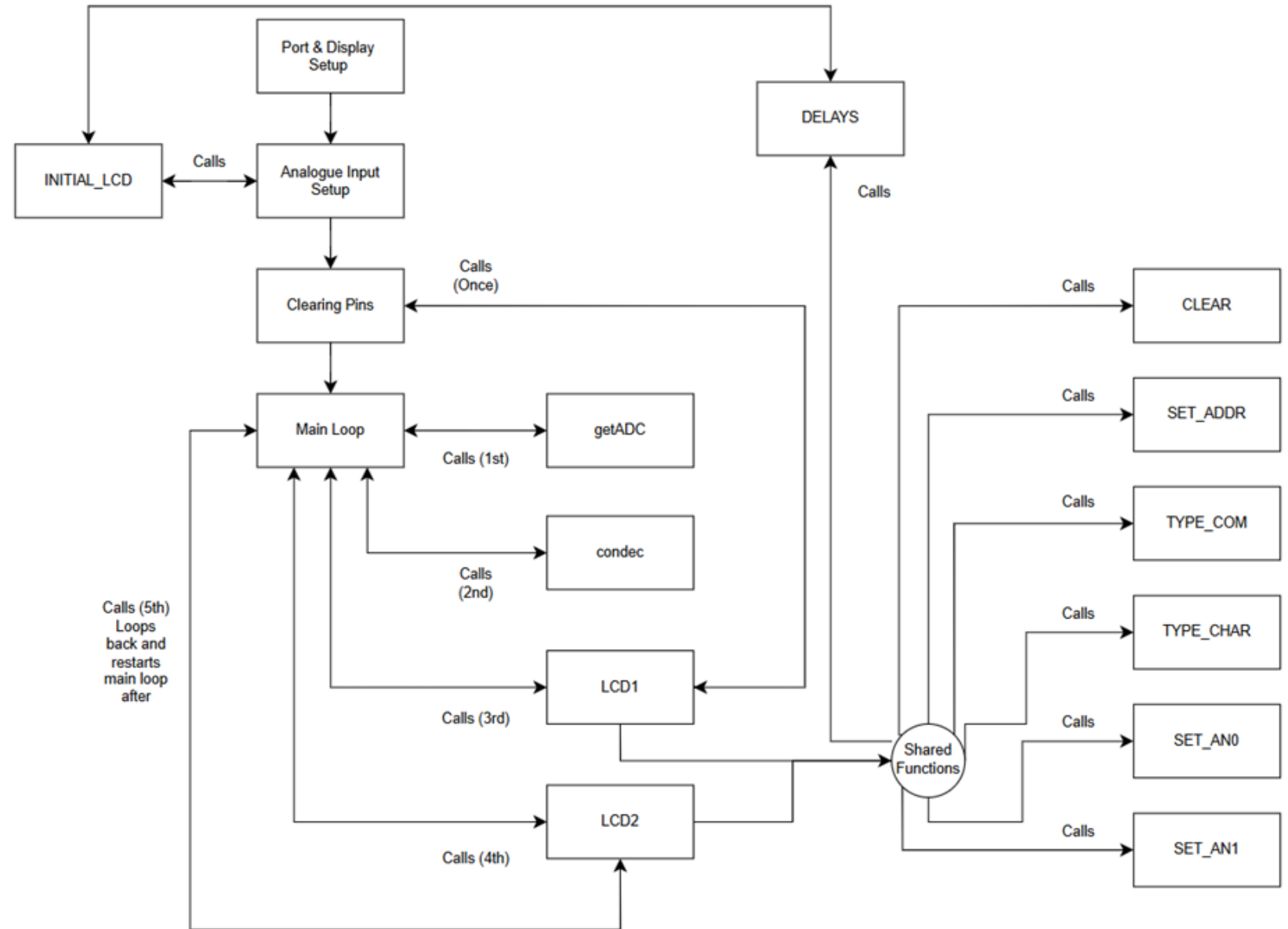
Change the value

$$f_c = 1/2\pi RC$$

The frequency can be calculated through this formula. from the image, the larger resistance, the better the filtering effect. However, excessive resistance will cause the amplitude to decrease.



Program Structure



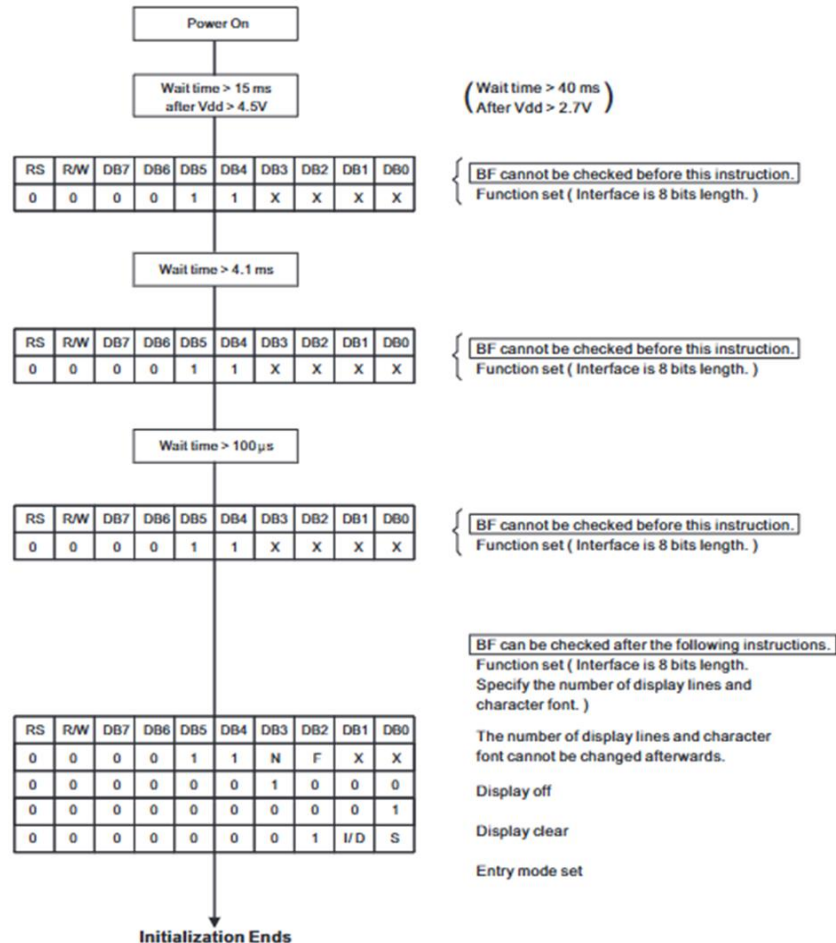
Global Functions

- CLEAR - To clear the LCD Display
- SETADDR - To go to the next line on the LCD Display
- TYPE_COM - To send a command to the LCD display
- TYPE_CHAR - To output a character to the LCD display
- SET_AN0 and SET_AN1 - To switch channels
- Delay (15ms) - To cause a 15ms delay

Initialisation of LCD Display

15. INITIALIZATION

15.1 8-bit interface mode (Condition: fosc = 270KHZ)



(FORDATA,2005)

code

INITIAL_LCD

```

BCF PORTE,1 ;E=0
BCF PORTE,2 ;RS=0
MOVLW b'00110000' ;D7-D0: 0011XXXX
BSF PORTE,1 ;E=1
NOP
MOVWF PORTD
NOP ;WAIT FOR 500ns
BCF PORTE,1 ;E=0

;WAIT FOR 15ms
CALL WAIT_15ms

BCF PORTE,1 ;E=0
BCF PORTE,2 ;RS=0
MOVLW b'00110000' ;D7-D0: 0011XXXX
BSF PORTE,1 ;E=1
NOP
MOVWF PORTD
NOP ;WAIT FOR 500ns
BCF PORTE,1 ;E=0

;WAIT FOR 15ms
CALL WAIT_15ms
BCF PORTE,1 ;E=0
BCF PORTE,2 ;RS=0
BCF PORTE,2 ;RS=0
MOVLW b'00110000' ;D7-D0: 0011XXXX
BSF PORTE,1 ;E=1
NOP
MOVWF PORTD
NOP ;WAIT FOR 500ns
BCF PORTE,1 ;E=0

```

```

; STEP 1 - FUNCTION SET (8 LINE)
CALL WAIT_15ms ; delay for 15ms
BCF PORTE,1 ; E=0
BCF PORTE,2 ; RS=0
MOVLW b'00110000' ; D7-D0: 0011XXXX
BSF PORTE,1 ; E=1
NOP
MOVWF PORTD
NOP ; delay for 15ms
BCF PORTE,1 ; E=0

; STEP 2 - DISPLAY OFF
CALL WAIT_15ms ; delay for 15ms
BCF PORTE,1 ; E=0
BCF PORTE,2 ; RS=0
MOVLW b'00001000' ; D7-D0: 00001000
BSF PORTE,1 ; E=1
NOP
MOVWF PORTD
NOP ; delay for 500ns
BCF PORTE,1 ; E=0

; STEP 3 - DISPLAY CLEAR
CALL WAIT_15ms ; delay for 15ms
BCF PORTE,1 ; E=0
BCF PORTE,2 ; RS=0
MOVLW b'00000001' ; D7-D0: 00000001
BSF PORTE,1 ; E=1
NOP
MOVWF PORTD
NOP ; delay for 500ns
BCF PORTE,1 ; E=0

; STEP 4 - ENTRY MODE SET
CALL WAIT_15ms ; delay for 15ms
BCF PORTE,1 ; E=0
BCF PORTE,2 ; RS=0
MOVLW b'00000110' ; D7-D0: 00000110
BSF PORTE,1 ; E=1
NOP
MOVWF PORTD
NOP ;WAIT FOR 500ns
BCF PORTE,1 ; E=0

; STEP 5 - DISPLAY ON
CALL WAIT_15ms ; delay for 15ms
BCF PORTE,1 ; E=0
BCF PORTE,2 ; RS=0
MOVLW b'00001100' ; D7-D0: 00001111
BSF PORTE,1 ; E=1
NOP
MOVWF PORTD
NOP ; delay for 500ns
BCF PORTE,1 ; E=0
CALL WAIT_15ms
GOTO FINISH

```

Firmware Configuration

```
; PIC16F877A Configuration Bit Settings: __config 0xFF3A
__CONFIG _FOSC_HS & _WDTE_OFF & _PWRTE_OFF & _BOREN_OFF & _LVP_OFF & _CPD_OFF & _WRT_OFF & _CP_OFF
```

Field	Option	Category	Setting
FOSC	HS	Oscillator Selection bits	HS oscillator
WDTE	OFF	Watchdog Timer Enable bit	WDT disabled
PWRTE	OFF	Power-up Timer Enable bit	PWRT disabled
BOREN	OFF	Brown-out Reset Enable bit	BOR disabled
LVP	OFF	Low-Voltage (Single-Supply) In-Circuit Serial Programming Enable bit	RB3 is digital I/O, HV on MCLR must be used for programming
CPD	OFF	Data EEPROM Memory Code Protection bit	Data EEPROM code protection off
WRT	OFF	Flash Program Memory Write Enable bits	Write protection off; all program memory may be written to by EECON control
CP	OFF	Flash Program Memory Code Protection bit	Code protection off

ADC Input

```
getADC    ; Read ADC input and store .....
wait      BSF      ADCON0,GO    ; start ADC..
          BTFSC    ADCON0,GO    ; ..and wait for finish
          GOTO     wait
          MOVF     ADRESH,W     ; store result high byte
          RETURN
```

BCD Conversion and Scaling

```
condec    ; Convert input to decimal .....
          MOVWF    ADbin        ; get ADC result
          CLRF     huns         ; zero hundreds digit
          CLRF     tens         ; zero tens digit
          CLRF     ones         ; zero ones digit
          ; Calculate hundreds.....
          BSF      STATUS,C     ; set carry for subtract
          MOVLW    D'100'       ; load 100
sub1       SUBWF    ADbin        ; and subtract from result
          INCF     huns         ; count number of loops
          BTFSC    STATUS,C     ; and check if done
          GOTO     sub1         ; no, carry on
          ADDWF    ADbin        ; yes, add 100 back on
          DECF     huns         ; and correct loop count
          ; Calculate tens digit.....
          BSF      STATUS,C     ; repeat process for tens
          MOVLW    D'10'        ; load 10
sub2       SUBWF    ADbin        ; and subtract from result
          INCF     tens         ; count number of loops
          BTFSC    STATUS,C     ; and check if done
          GOTO     sub2         ; no, carry on
          ADDWF    ADbin        ; yes, add 100 back on
          DECF     tens         ; and correct loop count
          MOVF     ADbin,W      ; load remainder
          MOVWF    ones         ; and store as ones digit
          RETURN                ; done
```

Switch Channel

To switch between AN0 and AN1 we need to move a certain 8-bit binary number to the ADCON0 register.

To change the channel depends on what the bit is between bit 5-3.

The other numbers are to select what register we using and if it is on or off.

```
SET_AN0
    MOVLW    B'01000001'
    MOVWF    ADCON0
    RETURN

SET_AN1
    MOVLW    B'01001001'
    MOVWF    ADCON0
    RETURN
```

REGISTER 11-1: ADCON0 REGISTER (ADDRESS 1Fh)

R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	R/W-0	U-0	R/W-0
ADCS1	ADCS0	CHS2	CHS1	CHS0	GO/DONE	—	ADON
bit 7				bit 0			

bit 7-8 ADCS1:ADCS0: A/D Conversion Clock Select bits (ADCON0 bits in bold)

ADCON1 <ADC S2>	ADCON0 <ADC S1:ADC S0>	Clock Conversion
0	00	<i>F_{osc}/2</i>
0	01	<i>F_{osc}/8</i>
0	10	<i>F_{osc}/32</i>
0	11	<i>F_{int}</i> (clock derived from the internal A/D RC oscillator)
1	00	<i>F_{osc}/4</i>
1	01	<i>F_{osc}/16</i>
1	10	<i>F_{osc}/64</i>
1	11	<i>F_{int}</i> (clock derived from the internal A/D RC oscillator)

bit 5-3 CHS2:CHS0: Analog Channel Select bits

000 = Channel 0 (AN0)
001 = Channel 1 (AN1)
010 = Channel 2 (AN2)
011 = Channel 3 (AN3)
100 = Channel 4 (AN4)
101 = Channel 5 (AN5)
110 = Channel 6 (AN6)
111 = Channel 7 (AN7)

Input Selection

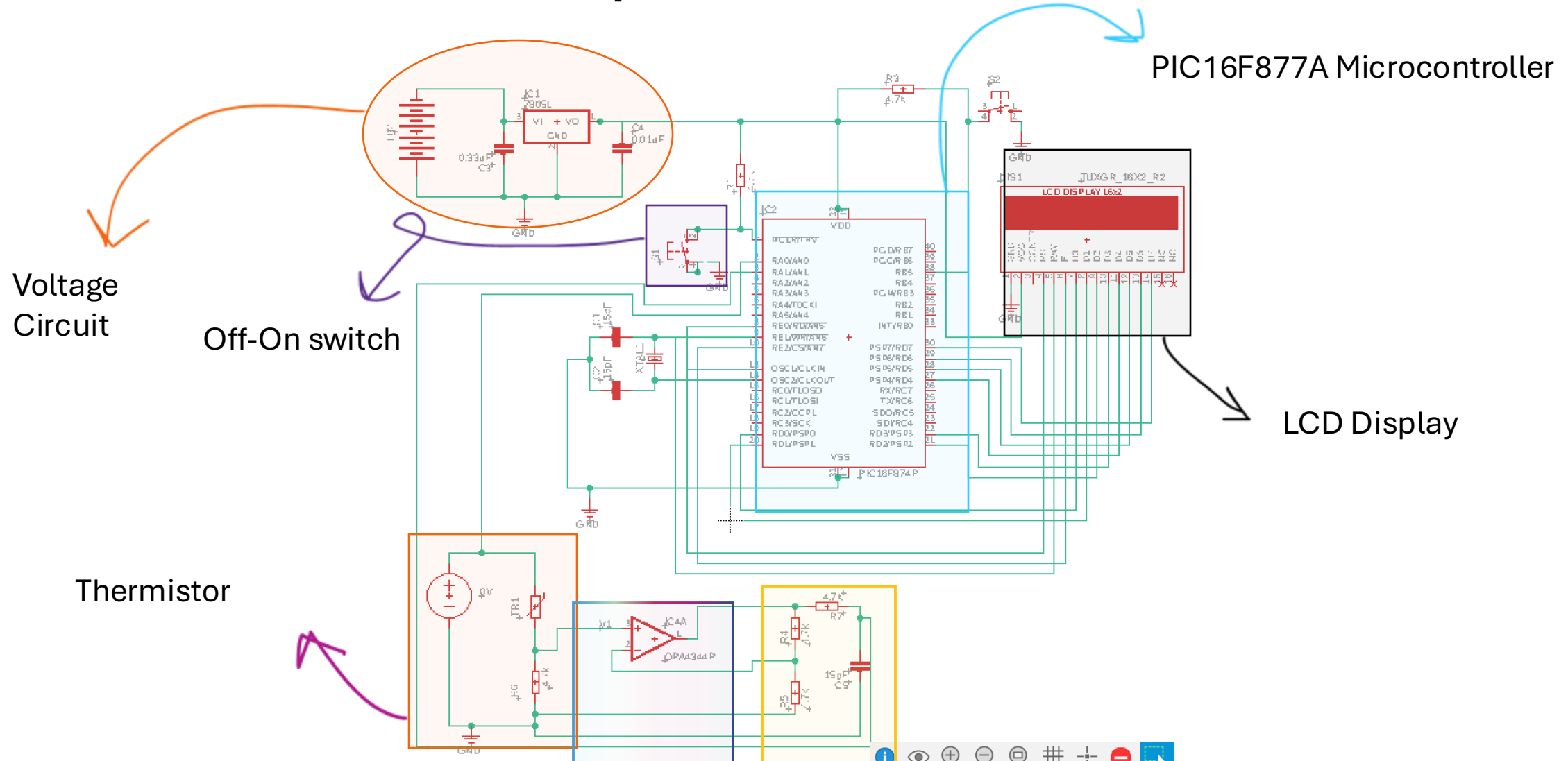
We chose to use RB0 as our way to switch channel if RB0 is 1 then the channel will be AN1, if RB0 becomes 0 then the channel becomes AN0.

```
BTFSS    PORTB, RB0  
CALL     CLEAR  
BTFSS    PORTB, RB0  
CALL     SET_AN0
```

Schematic Drawings in Fusion 360

- AFE Circuit
- Oscillating Circuit
- Power Regulation Circuit
- Reset Circuit
- LCD Display
- Microcontroller Circuit

Schematic Components

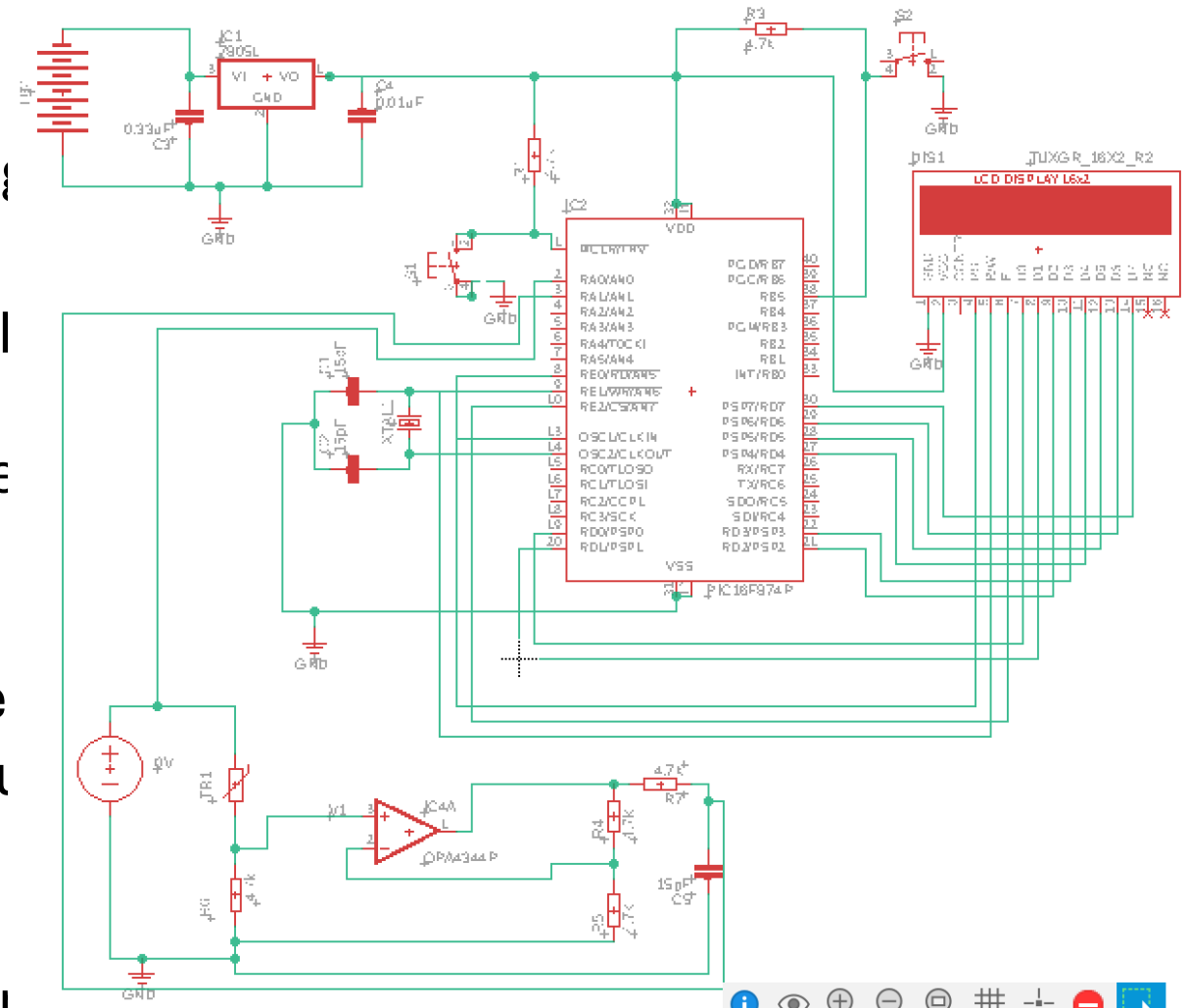


Microcontroller Connectivity

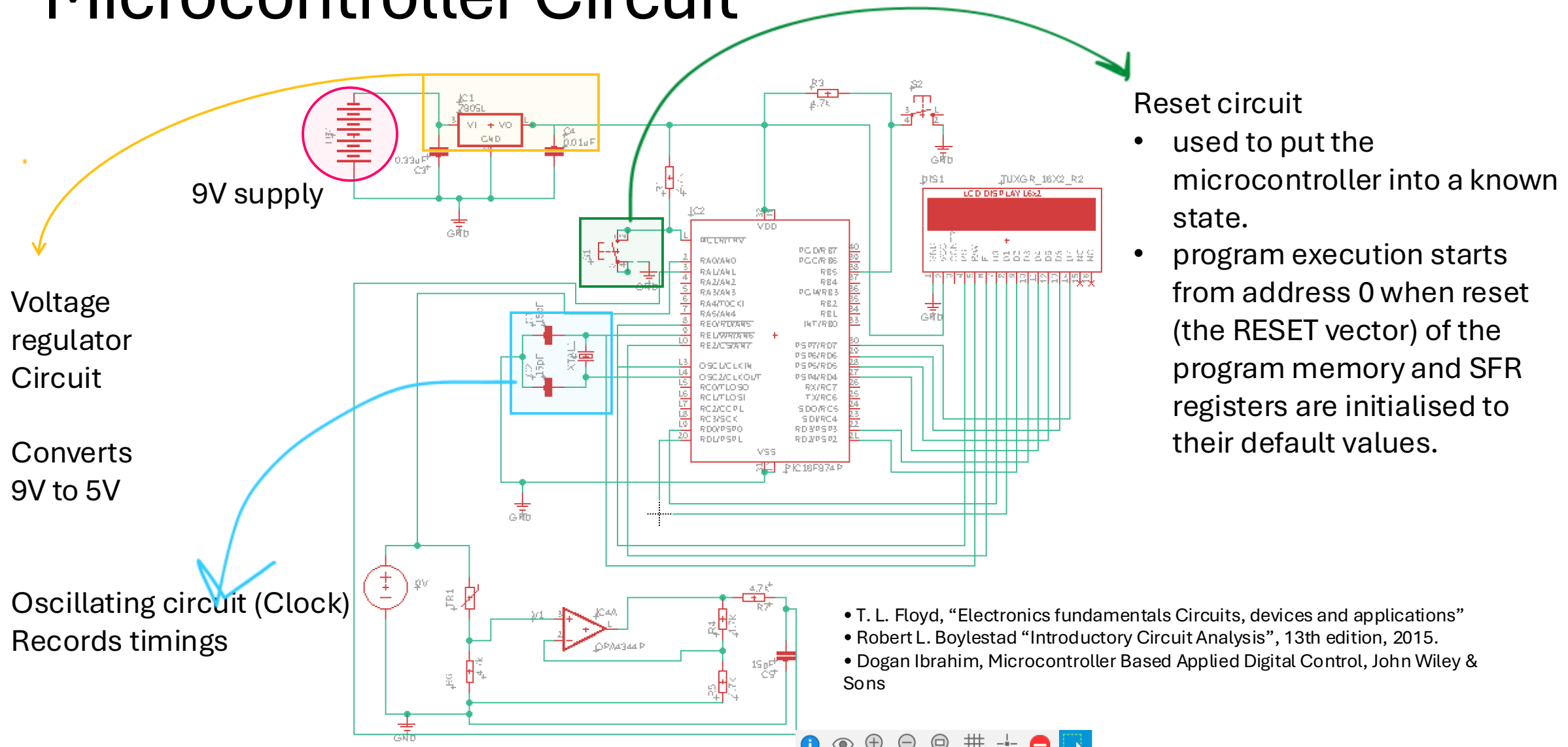
We have a 40-Pin microcontroller that:

- Has 8 Inputs for the registers
- Is connected to the LCD display through 30 to D0-D7
- Has VDD connected to the power supply VSS to GND
- Is connected to the AFE circuit that we through AN0 and AN1 for each thermistor/channel
- Has the MCLR pin connected to the re
- Has pin 13 (for input) and pin 14 (for ou
- connected to the oscillator circuit

(Microchip, 2000,



Microcontroller Circuit



LCD Display

We used a HD4478 16x2 LCD Display

Each pin has function which helps us display what we need

- Pin 4 is how we control whether we are sending a command or character to the display
 - Pin 5 lets us write to the display
 - Pin 6 enables the LCD to do something
 - Pin 7-14 is how we send the data (8-bit)
- (FORDATA, 2005)

8.PIN ASSIGNMENT

PIN	SYMBOL	FUNCTION
1	Vss	GND
2	Vdd	Power supply for LCM (+3.0V)
3	NC	No Connection
4	RS	Register Select Signal
5	R/W	Data Read / Write
6	E	Enable Signal
7-14	DB0 - DB7	Data bus line
15	NC	No Connection
16	NC	No connection

Input Select Mechanism

For our inputs, we had chosen to use RB0 which act as our physical button that will make the LCD display switch between thermistor 1 and 2.

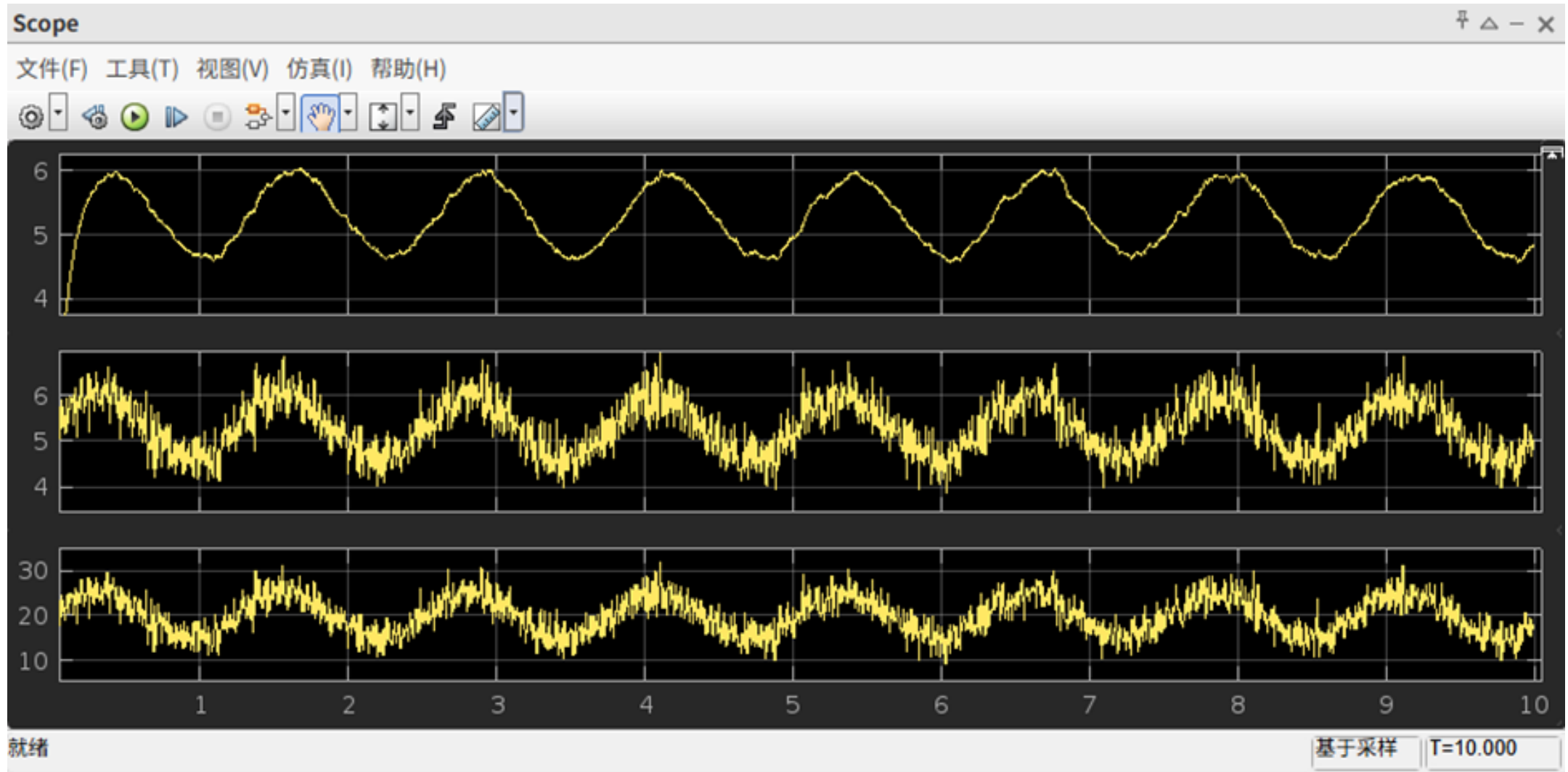
We also require a physical switch for the MCLR (pin 1 on PIC) which is connected to the reset circuit and the button that we have chosen for RB0 and MCLR pin is a MOSFET electronic switch which lets us control the channel being read and lets us reset the circuit (Ravi Teja, 2024).

AN0 and AN1 (Pin 2 and 3) our input between the AFE circuit and the microcontroller.

Testing Methodology

- Used breakpoints in MPLABX alongside tools such as the I/O Pins, stopwatch and SFRs to see changes as the program ran
- Export a Hex File from our program in MPLABX to simulate a board on PICSIM to see how our program operated
- Ran a behavioural simulation in MATLAB for the AFE circuit

AFE Behavioural Simulation Results



MPLABX Results

```
246      inid      ; initialise the LCD
                CALL    WAIT_15ms
                CALL    INITIAL_LCD
                RETURN

250
251      SET_AN0
252          MOVLW    B'01000001'
253          MOVWF    ADCON0
254          RETURN
255
256      SET_AN1
257          MOVLW    B'01001001'
258          MOVWF    ADCON0
259          RETURN
260
261      END
262
```

Stopwatch

- Target halted. Stopwatch cycle count = 465 (465 µs)
- Target halted. Stopwatch cycle count = 467 (467 µs)
- Target halted. Stopwatch cycle count = 468 (468 µs)
- Target halted. Stopwatch cycle count = 469 (469 µs)
- Target halted. Stopwatch cycle count = 471 (471 µs)
- Target halted. Stopwatch cycle count = 472 (472 µs)
- Target halted. Stopwatch cycle count = 473 (473 µs)
- Target halted. Stopwatch cycle count = 475 (475 µs)
- Target halted. Stopwatch cycle count = 29913 (29.913 ms)

MPLABX Results

```

org    0x00      ; Default start address
NOP    ; required for ICD mode
; Port & display setup.....
BANKSEL TRISC    ; Select bank 1
CLRF    TRISD    ; Display port is output
movlw   0x0f
movwf   TRISA
CLRF    TRISE    ; Display port is output
MOVLW   B'00000010' ; Analogue input setup code
MOVWF   ADCON1    ; Left justify result,
; Port A = analogue inputs
BANKSEL PORTC    ; Select bank 0
CLRF    PORTD    ; Clear display outputs

```

ess /	Name	Hex	Decimal	Binary	Char
	CCPR2H	0x00	0	00000000	'.'
	CCP2CON	0x00	0	00000000	'.'
	ADRESH	0x00	0	00000000	'.'
	ADCON0	0x00	0	00000000	'.'
	OPTION_REG	0xFF	255	11111111	'y'
	TRISA	0x3F	63	00111111	'?'
	TRISB	0xFF	255	11111111	'y'
	TRISC	0xFF	255	11111111	'y'
	TRISD	0xFF	255	11111111	'y'
	TRISE	0x07	7	00000111	'.'
	PIE1	0x00	0	00000000	'.'
	PIE2	0x00	0	00000000	'.'
	PCON	0x00	0	00000000	'.'
	SSPCON2	0x00	0	00000000	'.'
	PR2	0xFF	255	11111111	'y'
	SSPAD	0x00	0	00000000	'.'
	SSPSTAT	0x00	0	00000000	'.'
	TXSTA	0x00	0	00000000	'.'
	SPBRG	0x00	0	00000000	'.'
	CMCON	0x07	7	00000111	'.'
	CVRCON	0x00	0	00000000	'.'
	ADRESL	0x00	0	00000000	'.'
	ADCON1	0x00	0	00000000	'.'
	EEDATA	0x00	0	00000000	'.'
	FEADR	0x00	0	00000000	'.'

```

; Port & display setup.....
BANKSEL TRISC    ; Select bank 1
CLRF    TRISD    ; Display port is output
movlw   0x0f
movwf   TRISA
CLRF    TRISE    ; Display port is output
MOVLW   B'00000010' ; Analogue input setup code
MOVWF   ADCON1    ; Left justify result,
; Port A = analogue inputs
BANKSEL PORTC    ; Select bank 0
CLRF    PORTD    ; Clear display outputs
CLRF    PORTE    ; Clear display outputs
BTFSS   PORTB, RB1
MOVLW   B'01000001' ; Analogue input setup code
MOVWF   ADCON0    ; f/8, RA0, done, enable
CALL    inid      ; Initialise the display
; Clearing Pins
BCF     PORTB, RB0

```

ss /	Name	Hex	Decimal	Binary	Char
	CCPR2H	0x00	0	00000000	'.'
	CCP2CON	0x00	0	00000000	'.'
	ADRESH	0x00	0	00000000	'.'
	ADCON0	0x41	65	01000001	'A'
	OPTION_REG	0xFF	255	11111111	'y'
	TRISA	0x0F	15	00001111	'.'
	TRISB	0xFF	255	11111111	'y'
	TRISC	0xFF	255	11111111	'y'
	TRISD	0x00	0	00000000	'.'
	TRISE	0x00	0	00000000	'.'
	PIE1	0x00	0	00000000	'.'
	PIE2	0x00	0	00000000	'.'
	PCON	0x00	0	00000000	'.'
	SSPCON2	0x00	0	00000000	'.'
	PR2	0xFF	255	11111111	'y'
	SSPAD	0x00	0	00000000	'.'
	SSPSTAT	0x00	0	00000000	'.'
	TXSTA	0x00	0	00000000	'.'
	SPBRG	0x00	0	00000000	'.'
	CMCON	0x07	7	00000111	'.'
	CVRCON	0x00	0	00000000	'.'
	ADRESL	0x00	0	00000000	'.'
	ADCON1	0x00	0	00000000	'.'

MPLABX Results

73	CALL	LCD1	; display input
74	GOTO	start	; jump to main loop
75	; SUBROUTINES ;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;;		
76	; Read ADC input and store		
77	getADC	BSF	ADCON0,GO ; start ADC..
78	wait	BTFSC	ADCON0,GO ; ..and wait for finish
79		GOTO	wait
80		MOVF	ADRESH,W ; store result high byte
81		RETURN	
82	; Convert input to decimal		
83	condec	MOVWF	ADbin ; get ADC result
84		CLRF	huns ; zero hundreds digit
85		CLRF	tens ; zero tens digit
86		CLRF	ones ; zero ones digit
87	; Calculate hundreds.....		
88		BSF	STATUS,C ; set carry for subtract
89		MOVLW	D'100' ; load 100
90	sub1	SUBWF	ADbin ; and subtract from result
91		INCF	huns ; count number of loops
92		BTFSC	STATUS,C ; and check if done
93		GOTO	sub1 ; no, carry on
94		ADDWF	ADbin ; yes, add 100 back on
95		DECF	huns ; and correct loop count
96	; Calculate tens digit.....		
97		BSF	STATUS,C ; repeat process for tens
98		MOVLW	D'10' ; load 10
99	sub2	SUBWF	ADbin ; and subtract from result
100		INCF	tens ; count number of loops
101		BTFSC	STATUS,C ; and check if done
102		GOTO	sub2 ; no, carry on
103		ADDWF	ADbin ; yes, add 100 back on
104		DECF	tens ; and correct loop count
105		MOVF	ADbin,W ; load remainder
106		MOVWF	ones ; and store as ones digit
107		RETURN	; done
108			
109	LCD1		
110		BTFSS	PORTB,RB0
111		CALL	SET_AN0
112		RETURN	
113	; Output to display.....		
114	; End of main program.....		

I/O Pins		
Pin	Mode	Value
RB0	Din	0

MPLABX Results

```
182
183 LCD2
184     BTFSS    PORTB, RB0
185     CALL     SET_AN1
186     ; Output to display.....
187     MOV LW   080      ; code to home cursor
188     MOVWF    cmd_reg
189     CALL     TYPE_COM  ; output it to display
190     ; Convert digits to ASCII and display.....
191     MOV LW   030      ; load ASCII offset
192     ADDWF    huns      ; convert hundreds to ASCII
193     ADDWF    tens      ; convert tens to ASCII
194     ADDWF    ones      ; convert ones to ASCII
195     MOVF     huns, W    ; load hundreds code
196     MOVWF    lcd_reg
197     CALL     TYPE_CHAR ; and send to display
198     MOV LW   '.'       ; load point code
199     MOVWF    lcd_reg
200     CALL     TYPE_CHAR ; and output
```

I/O Pins

Pin	Mode	Value
RB0	Din	1

PICSIM Results



Conclusion

Overall, we have managed to layout the plans and schematics for the project. In addition to this we have created software that works and will send the messages we want to the LCD display. However, we still need to improve our code as the channels do not switch properly. Also, our PCB design is wrong as there is only one AFE circuit in schematic.

In the future we hope to correct these errors as well as build the actual hardware for this. Improvement to this project include adding more thermistors for more areas to measure, more channels or a bigger LCD display (16x4).

References

dwyeromega *Thermistor*. Available at: <https://www.dwyeromega.com/en-us/resources/thermistor>

Arrow (2023) *Basics of Analog to Digital Converters*. Available at: <https://www.arrow.com/en/research-and-events/articles/engineering-resource-basics-of-analog-to-digital-converters>

Morris, A.S. (2001) *Measurement and Instrumentation Principles* Chantilly: Elsevier Science & Technology.

Park, J. and Mackay, S. (2003) *Practical data acquisition for instrumentation and control systems* Newnes, pp. 1–2.

Microchip (2003) *PIC16F877A Datasheet*. Available at: <https://ww1.microchip.com/downloads/en/devicedoc/39582b.pdf>

FORDATA (2005) *LCD Datasheet*. Available at: <https://www.farnell.com/datasheets/653650.pdf>

Ravi Teja (2024) *Switches*. Available at: <https://www.electronicshub.org/switches/>